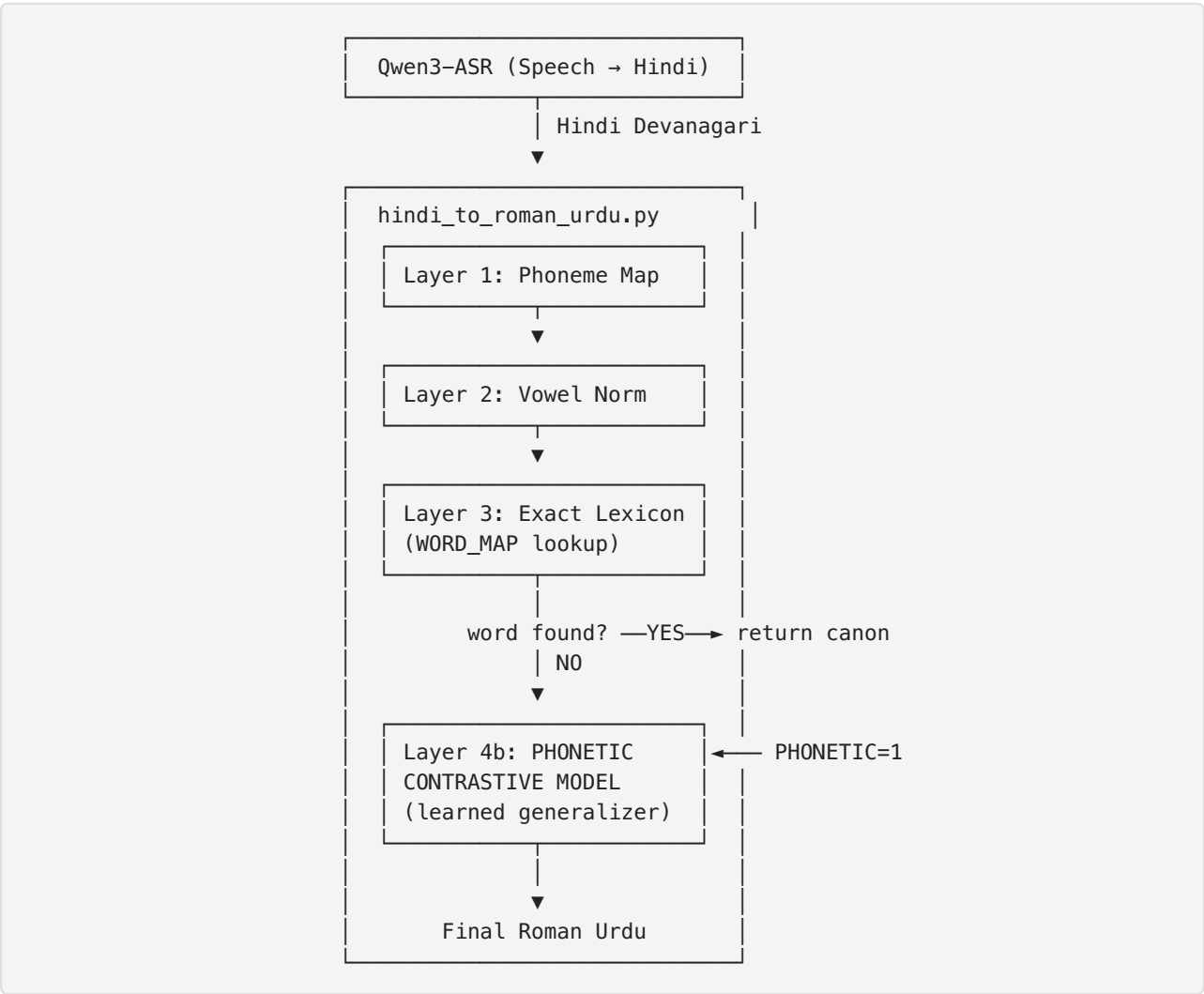


Phonetic Contrastive Model – Architecture Reference Diagram

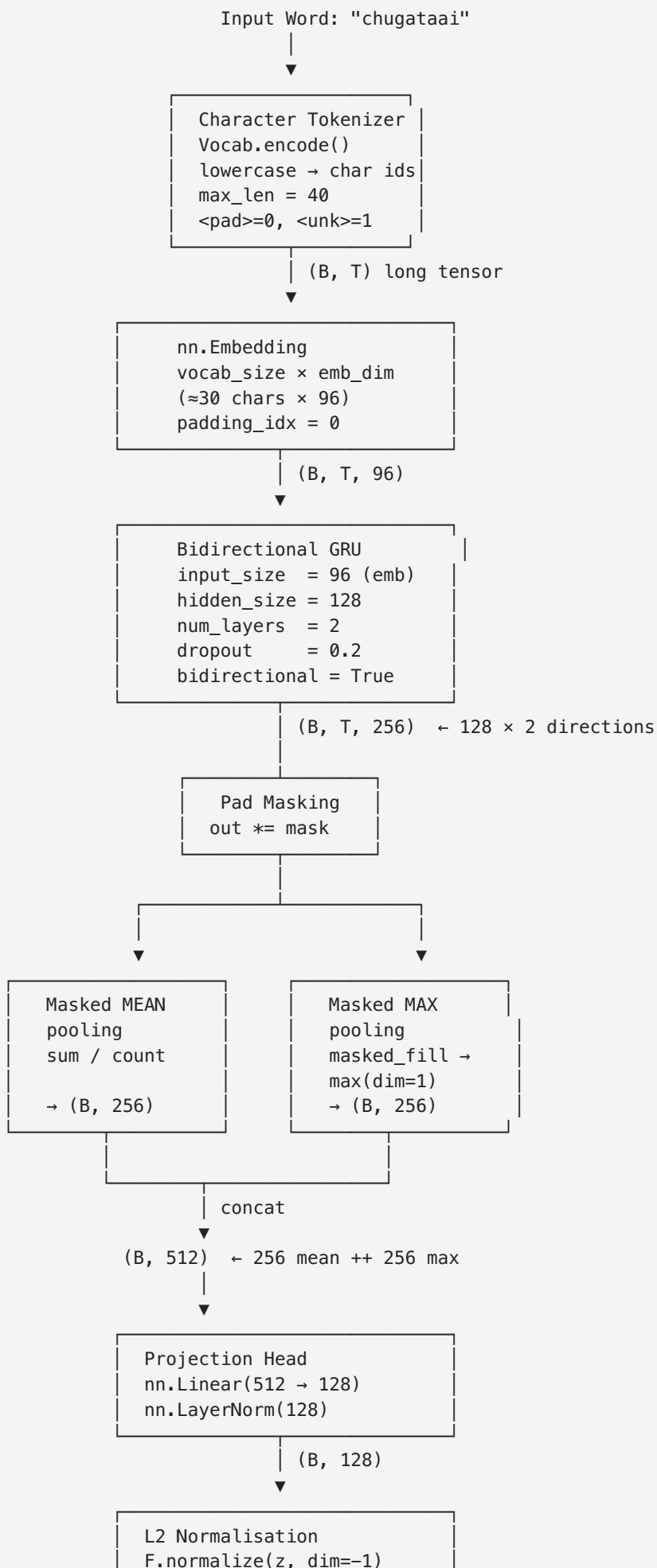
Module: `qwen3-asr-local/phonetic_contrastive_model/` **Checkpoint:** `phonetic_contrastive_model/models/phonetic_contrastive_v1.pt` **Status:** production-ready, integrated as optional Layer 4b in `hindi_to_roman_urdu.py`

1. System-Level Position in the ASR Pipeline



Gating: Env var `PHONETIC=1` enables the model. If disabled, falls back to the deprecated rule-based resolver or returns the word unchanged.

2. Model Architecture — CharEncoder (Siamese Bi-Encoder)



(B, 128) unit-norm embeddings

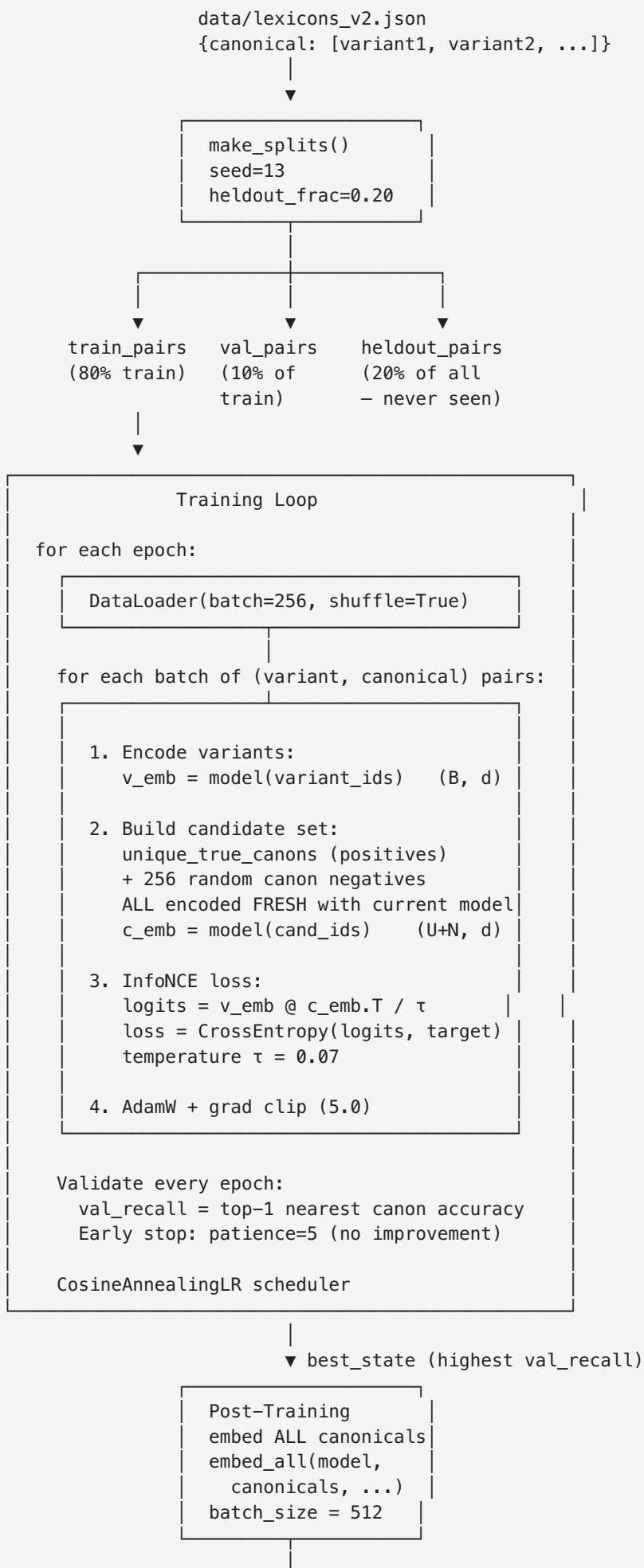
2.1 Hyperparameters (default config)

Parameter	Value	Description
emb_dim	96	Character embedding dimension
hidden	128	GRU hidden size (per direction)
out_dim	128	Final embedding dimension
num_layers	2	Stacked GRU layers
dropout	0.2	Between GRU layers
pooled_dim	512	128×2 (bi) $\times 2$ (mean+max)
vocab_size	~30	Lowercase a-z + <pad> + <unk>

2.2 Key Design Decisions

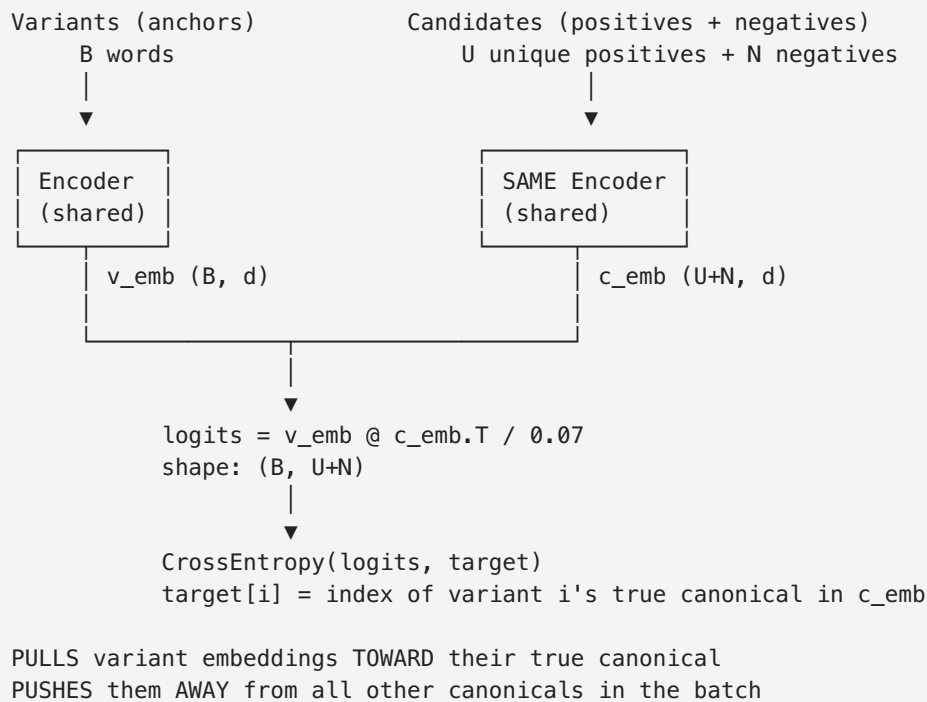
- **Character-level** — the signal is phonetic/spelling ($z \leftrightarrow j$, $aa \leftrightarrow a$, $ee \leftrightarrow i$), not semantic
- **Siamese architecture** — the SAME encoder embeds both variants and canonicals into one shared space
- **Mean + Max pooling** — length-invariant, no reliance on a final hidden state
- **LayerNorm** on projection — stable embeddings across diverse inputs
- **L2-normalised** output — cosine similarity = dot product

3. Training Pipeline



Save Checkpoint
phonetic_contrastive
_v1.pt

3.1 InfoNCE Loss — How It Works



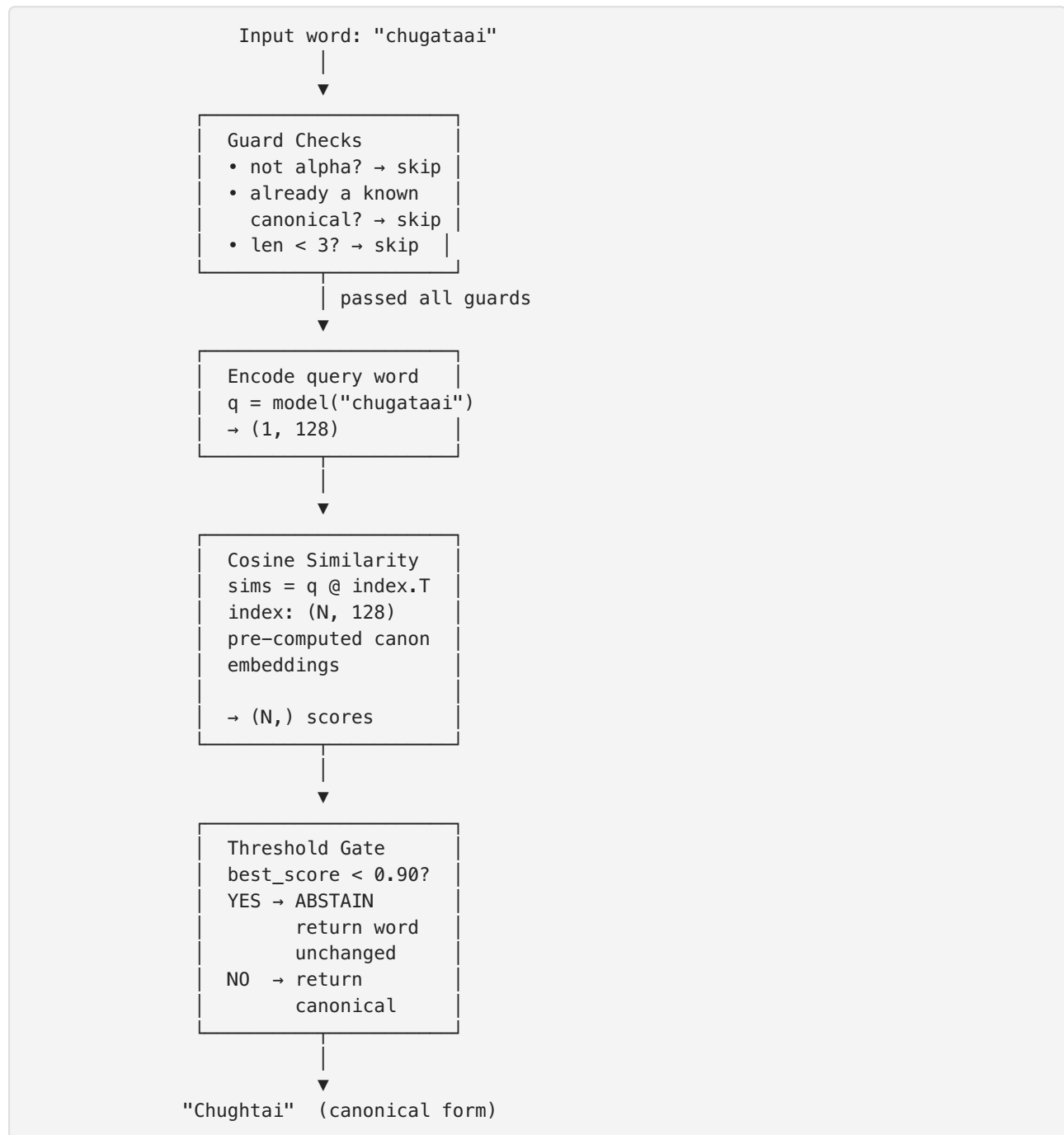
Why unique canonicals? If two variants share a canonical (e.g. "siddiqi" and "siddiqui" both → "Siddiqui"), treating both canonical copies as negatives of each other is a false negative. Using unique canonicals avoids this.

3.2 Checkpoint Contents

```
torch.save({
    "state_dict":          model weights (CPU),
    "itos":                 character vocabulary list,
    "config": {
        "emb": 96, "hidden": 128,
        "out": 128, "layers": 2, "temp": 0.07
    },
    "canonicals":           list of all canonical strings,
    "canonical_embeddings": (N, 128) pre-computed embeddings,
    "meta": {
        "max_epochs", "patience", "seed",
        "train_pairs", "best_val_recall"
    }
})
```

Everything needed to reconstruct the corrector is in one file — no external state.

4. Inference Pipeline — PhoneticContrastiveCorrector



4.1 Abstain Mechanism — The Safety Guard

ABSTAIN LOGIC

Three conditions cause the model to LEAVE a word unchanged:

1. `word.isalpha() == False` → non-alphabetic, skip
2. `word in known_canonicals` → already correct
3. `cosine_sim < threshold` → not confident enough
(default threshold = 0.90)

This prevents the model from "correcting" real words that happen to look like garbles of something else.

Example: "apareshan" (score 0.78) → left as-is

4.2 Runtime Complexity

Operation	Cost
Encode 1 word	1 forward pass (~0.1ms)
Nearest-neighbor	1 matmul ($1 \times N$)
Total per word	$O(T + N)$ where T =word length, N =canonicals
Index load	Once at startup

5. Canonical Index Management

Pre-computed at training time:

```
for each canonical in lexicon:
    emb = model.encode(canonical)
canonical_embeddings = stack(all embs) → (N, 128)
L2-normalised → cosine sim = dot product
```

Stored in checkpoint → loaded once at startup

add_canonical(new_term) – runtime onboarding:

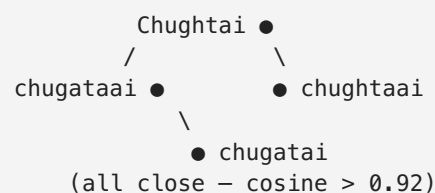
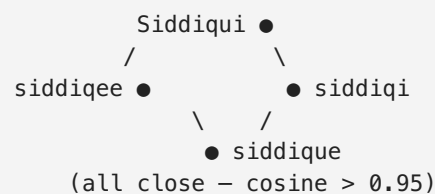
```
emb = model.encode(new_term)
index = cat([index, normalize(emb)]) → (N+1, 128)
canonicals.append(new_term)
No retraining required!
```

rebuild_index(new_list) – full re-index (idempotent):

Re-encode ALL canonicals with current model weights.
Used by extend_canonicals.py and prune_lexicon.py
Deterministic: same input → same output

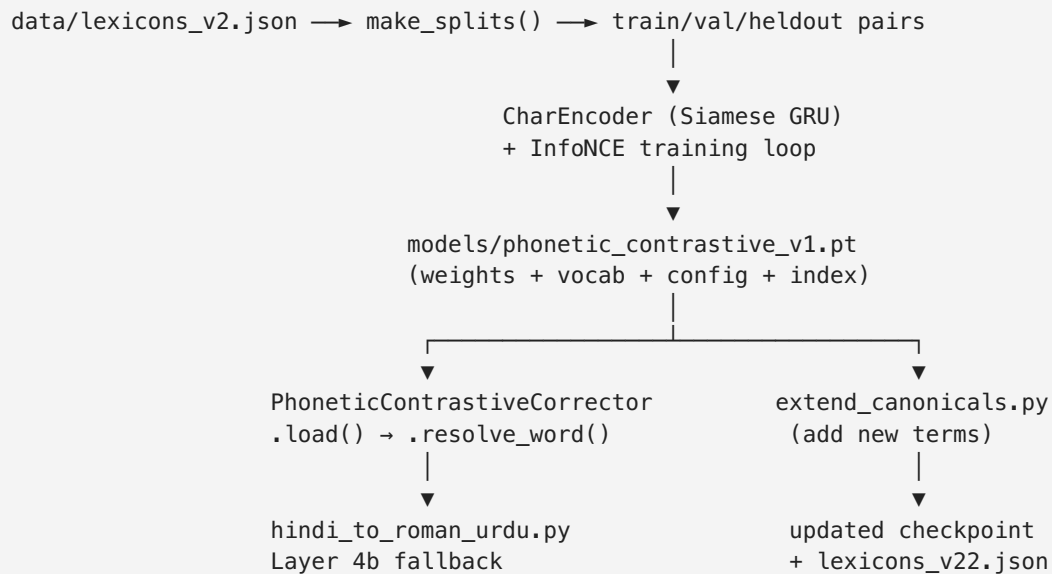
6. Embedding Space Geometry (Conceptual)

128-dim unit hypersphere



● area
(far from both clusters)
cosine < 0.50 to any name
→ ABSTAIN, left unchanged

7. Data Flow Summary



8. Evaluation Metrics (4-number scorecard)

1. HELD-OUT VARIANT RECALL
Unseen spellings → correct canonical? (top-1, top-3)
The GENERALISATION number
2. ABSTAIN SAFETY
Real gold words (not canonicals) → % left UNCHANGED
The ANTI-CORRUPTION number
3. EXACT-NAME HELD-OUT
Entity-name variants (Siddique → Siddiqui) → exact match
The DECISIVE test
4. 80-CALL diff_words
End-to-end production accuracy: v2 baseline vs v2 + model
The PRODUCTION-METRIC number

9. Integration with Transliteration Pipeline

```
# hindi_to_roman_urdu.py – Layer 4b activation

_PHONETIC = None
if os.getenv('PHONETIC') in ('1', 'true', 'yes', 'on'):
    _PHONETIC = PhoneticContrastiveCorrector.load(
        threshold=float(os.getenv('PHONETIC_THRESHOLD', '0.90'))
    )

# In fix_word() – only runs when exact lexicon misses:
def fix_word(word):
    canon = WORD_MAP.get(word.lower())
    if not canon:
        if _PHONETIC is not None:
            return _PHONETIC.resolve_word(word)    # ← learned fallback
        if _RESOLVER is not None:
            return _RESOLVER.resolve_word(word)    # ← deprecated rule-based
    return word
return canon
```

Priority chain: Exact lexicon → Phonetic model → Resolver → unchanged

10. File Map

File	Purpose
model.py	CharEncoder (GRU bi-encoder) + info_nce loss
data.py	Vocab, pad_batch, make_splits, load_pairs
train.py	Full training loop, checkpoint saving
corrector.py	PhoneticContrastiveCorrector — inference runtime
eval.py	4-metric evaluation scorecard
extend_canonicals.py	Add new canonicals without retraining
prune_lexicon.py	Remove low-utility lexicon entries
sweep.py	Hyperparameter sweep over thresholds
models/phonetic_contrastive_v1.pt	The production checkpoint (~3.2 MB)